

**Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №7
По дисциплине «Современные платформы программирования»**

Выполнил:
Студент 3 курса
Группы ПО-12
Ишимов А.В.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить возможности языка программирования Python в разработке оконных приложений.

Вариант №7

Задание 1. Построение графических примитивов и надписей

Изобразить в окне приложения отрезок, вращающийся в плоскости формы вокруг точки, движущейся по отрезку.

Код:

```
import tkinter as tk
import math
import time
from PIL import ImageGrab

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Segment:
    """Отрезок, заданный двумя точками."""
    def __init__(self, p1: Point, p2: Point):
        self.p1 = p1
        self.p2 = p2

    def point_at(self, t: float) -> Point:
        """Возвращает точку на отрезке при параметре  $t \in [0, 1]$ ."""
        return Point(
            self.p1.x + t * (self.p2.x - self.p1.x),
            self.p1.y + t * (self.p2.y - self.p1.y),
        )

    def length(self) -> float:
        dx = self.p2.x - self.p1.x
        dy = self.p2.y - self.p1.y
        return math.hypot(dx, dy)

class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Вращающийся отрезок")

        self.running = False
        self.t = 0.0          # Положение опорной точки на базовом отрезке
        [0..1]
        self.t_dir = 1        # Направление движения опорной точки
```

```

self.angle = 0.0      # Текущий угол вращения

self._create_canvas()
self._create_controls()
self._create_segment_inputs()
self._draw_base_segment()

# ----- UI -----
-

def _create_canvas(self):
    self.canvas = tk.Canvas(self.root, width=600, height=500, bg="white")
    self.canvas.pack()

def _create_controls(self):
    controls = tk.Frame(self.root)
    controls.pack(pady=5)

    tk.Label(controls, text="Длина вращ. отрезка:").grid(row=0, column=0)
    self.len_entry = tk.Entry(controls, width=5)
    self.len_entry.insert(0, "80")
    self.len_entry.grid(row=0, column=1)

    tk.Label(controls, text="Скорость:").grid(row=0, column=2)
    self.speed = tk.Scale(controls, from_=1, to=100,
orient=tk.HORIZONTAL, length=150)
    self.speed.set(50)
    self.speed.grid(row=0, column=3)

    tk.Button(controls, text="Старт",      command=self.start).grid(row=0,
column=4)
    tk.Button(controls, text="Пауза",      command=self.pause).grid(row=0,
column=5)
    tk.Button(controls, text="Скриншот",
command=self.screenshot).grid(row=0, column=6)

def _create_segment_inputs(self):
    frame = tk.LabelFrame(self.root, text="Координаты базового отрезка")
    frame.pack(pady=5)

    tk.Label(frame, text="P1 (x, y)").grid(row=0, column=0)
    self.p1x = tk.Entry(frame, width=5); self.p1x.insert(0, "100")
    self.p1y = tk.Entry(frame, width=5); self.p1y.insert(0, "250")
    self.p1x.grid(row=0, column=1); self.p1y.grid(row=0, column=2)

    tk.Label(frame, text="P2 (x, y)").grid(row=0, column=3)
    self.p2x = tk.Entry(frame, width=5); self.p2x.insert(0, "500")
    self.p2y = tk.Entry(frame, width=5); self.p2y.insert(0, "250")
    self.p2x.grid(row=0, column=4); self.p2y.grid(row=0, column=5)

    tk.Button(frame, text="Обновить", command=self._update_segment).grid(
        row=0, column=6, padx=10
    )

# ----- helpers -----
--

def _read_segment(self) -> Segment:
    return Segment(
        Point(int(self.p1x.get()), int(self.p1y.get())),
        Point(int(self.p2x.get()), int(self.p2y.get())),
    )

def _update_segment(self):

```

```

        try:
            self.base_segment = self._read_segment()
            self._draw_base_segment()
        except ValueError:
            pass

def _draw_base_segment(self):
    try:
        seg = self._read_segment()
    except ValueError:
        return
    self.base_segment = seg
    self.canvas.delete("all")
    self.canvas.create_line(
        seg.p1.x, seg.p1.y, seg.p2.x, seg.p2.y,
        fill="black", width=2, tags="base"
    )

# ----- animation -----
--

def start(self):
    self.running = True
    try:
        self.base_segment = self._read_segment()
    except ValueError:
        return
    self.t = 0.0
    self.t_dir = 1
    self.angle = 0.0
    self._animate()

def pause(self):
    self.running = not self.running
    if self.running:
        self._animate()

def _animate(self):
    if not self.running:
        return

    # Шар зависит от скорости
    speed_val = self.speed.get() # 1..100
    t_step = speed_val * 0.0003 # скорость движения по базовому
    отрезку
    a_step = speed_val * 0.06 # скорость вращения (градусы за
    кадр)

    # Движение опорной точки туда-обратно
    self.t += t_step * self.t_dir
    if self.t >= 1.0:
        self.t = 1.0
        self.t_dir = -1
    elif self.t <= 0.0:
        self.t = 0.0
        self.t_dir = 1

    self.angle = (self.angle + a_step) % 360

    # Опорная точка на базовом отрезке
    pivot = self.base_segment.point_at(self.t)

    # Длина вращающегося отрезка
    try:

```

```

        rot_len = float(self.len_entry.get())
    except ValueError:
        rot_len = 80.0

    # Концы вращающегося отрезка
    rad = math.radians(self.angle)
    dx = rot_len * math.cos(rad)
    dy = rot_len * math.sin(rad)

    x1r = pivot.x - dx
    y1r = pivot.y - dy
    x2r = pivot.x + dx
    y2r = pivot.y + dy

    # Перерисовка
    self.canvas.delete("all")

    # Базовый отрезок
    seg = self.base_segment
    self.canvas.create_line(
        seg.p1.x, seg.p1.y, seg.p2.x, seg.p2.y,
        fill="black", width=2
    )

    # Вращающийся отрезок
    self.canvas.create_line(x1r, y1r, x2r, y2r, fill="blue", width=2)

    # Опорная точка
    r = 4
    self.canvas.create_oval(
        pivot.x - r, pivot.y - r, pivot.x + r, pivot.y + r,
        fill="red", outline="red"
    )

    # Следующий кадр: задержка обратно пропорциональна скорости
    delay = max(10, 110 - speed_val)
    self.root.after(delay, self._animate)

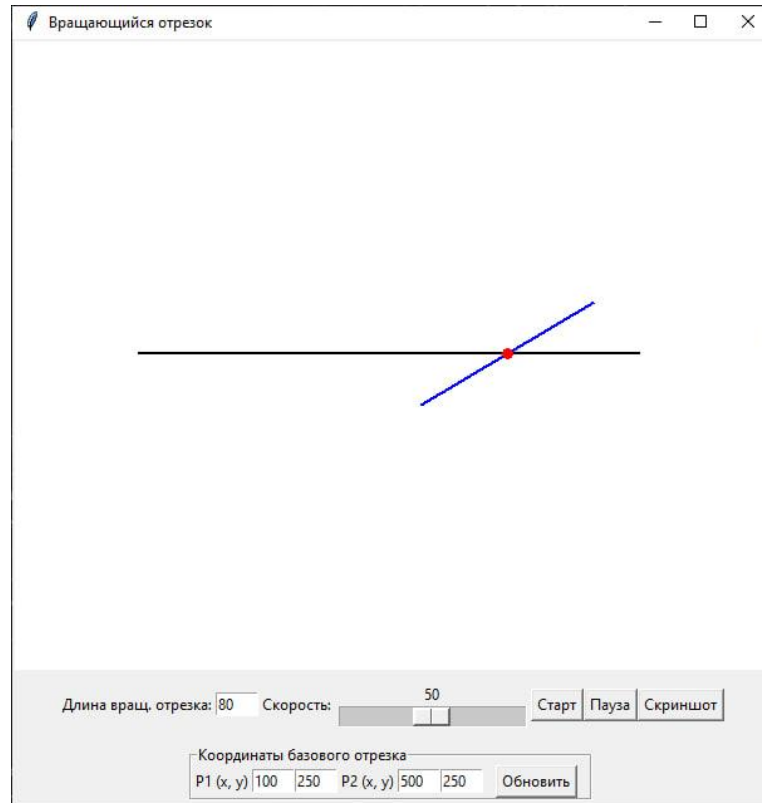
# ----- screenshot -----
--

def screenshot(self):
    self.canvas.update()
    x = self.canvas.winfo_rootx()
    y = self.canvas.winfo_rooty()
    x1 = x + self.canvas.winfo_width()
    y1 = y + self.canvas.winfo_height()
    filename = f"screenshot_segment_{int(time.time())}.png"
    ImageGrab.grab().crop((x, y, x1, y1)).save(filename)
    print(f"Скриншот сохранён: {filename}")

if __name__ == "__main__":
    main_root = tk.Tk()
    app = App(main_root)
    main_root.mainloop()

```

Результат:



Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

7) Снежинка Коха

Код:

```
import time
import tkinter as tk
from PIL import Image, ImageDraw, ImageTk, ImageGrab
import math

class KochSnowflake:
    """Генератор снежинки Коха."""

    def __init__(self, size: int, iterations: int):
        self.size = size
        self.iterations = iterations
```

```

def _koch_points(self, p1, p2, depth):
    """Рекурсивно строит список точек кривой Коха между p1 и p2."""
    if depth == 0:
        return [p1, p2]

    x1, y1 = p1
    x2, y2 = p2

    # Три точки деления отрезка
    ax = x1 + (x2 - x1) / 3
    ay = y1 + (y2 - y1) / 3

    bx = x1 + 2 * (x2 - x1) / 3
    by = y1 + 2 * (y2 - y1) / 3

    # Вершина равностороннего треугольника
    angle = math.atan2(y2 - y1, x2 - x1) - math.pi / 3
    length = math.hypot(x2 - x1, y2 - y1) / 3
    cx = ax + length * math.cos(angle)
    cy = ay + length * math.sin(angle)

    pts = []
    pts += self._koch_points(p1, (ax, ay), depth - 1)[:1]
    pts += self._koch_points((ax, ay), (cx, cy), depth - 1)[:1]
    pts += self._koch_points((cx, cy), (bx, by), depth - 1)[:1]
    pts += self._koch_points((bx, by), p2, depth - 1)
    return pts

def get_points(self, cx, cy):
    """Возвращает все точки снежинки Коха, центрированной в (cx, cy)."""
    r = self.size / 2

    # Три вершины равностороннего треугольника
    p1 = (cx + r * math.cos(math.radians(-90)),
          cy + r * math.sin(math.radians(-90)))
    p2 = (cx + r * math.cos(math.radians(30)),
          cy + r * math.sin(math.radians(30)))
    p3 = (cx + r * math.cos(math.radians(150)),
          cy + r * math.sin(math.radians(150)))

    pts = []
    pts += self._koch_points(p1, p2, self.iterations)[:1]
    pts += self._koch_points(p2, p3, self.iterations)[:1]
    pts += self._koch_points(p3, p1, self.iterations)[:1]
    pts.append(pts[0]) # замкнуть контур
    return pts

class KochApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Снежинка Коха")

        self.img = None
        self.photo = None

        self._create_canvas()
        self._create_controls()

    # ----- UI -----

    def _create_canvas(self):
        self.canvas = tk.Canvas(self.root, width=600, height=550, bg="white")

```

```

self.canvas.pack()

def _create_controls(self):
    controls = tk.Frame(self.root)
    controls.pack(pady=5)

    # Размер
    tk.Label(controls, text="Размер:").grid(row=0, column=0)
    self.size_entry = tk.Entry(controls, width=5)
    self.size_entry.insert(0, "400")
    self.size_entry.grid(row=0, column=1)

    # Итерации
    tk.Label(controls, text="Итерации (0-6):").grid(row=0, column=2)
    self.iter_entry = tk.Entry(controls, width=5)
    self.iter_entry.insert(0, "4")
    self.iter_entry.grid(row=0, column=3)

    # Цвет линии
    tk.Label(controls, text="Цвет:").grid(row=0, column=4)
    self.color_var = tk.StringVar(value="blue")
    color_menu = tk.OptionMenu(controls, self.color_var, "blue", "black",
"red", "green", "purple")
    color_menu.grid(row=0, column=5)

    # Заливка
    self.fill_var = tk.BooleanVar(value=True)
    tk.Checkbutton(controls, text="Заливка",
variable=self.fill_var).grid(row=0, column=6)

    # Кнопки
    tk.Button(controls, text="Нарисовать", command=self.draw).grid(row=0,
column=7, padx=5)
    tk.Button(controls, text="Скриншот",
command=self.screenshot).grid(row=0, column=8)

    # ----- drawing -----
-

def draw(self):
    try:
        size = int(self.size_entry.get())
        iterations = int(self.iter_entry.get())
        iterations = max(0, min(iterations, 6)) # ограничиваем 0..6
    except ValueError:
        return

    width = self.canvas.winfo_width()
    height = self.canvas.winfo_height()
    cx, cy = width // 2, height // 2

    snowflake = KochSnowflake(size, iterations)
    pts = snowflake.get_points(cx, cy)

    # Рисуем через PIL для сглаживания
    self.img = Image.new("RGB", (width, height), "white")
    draw = ImageDraw.Draw(self.img)

    flat = [(x, y) for x, y in pts]
    color = self.color_var.get()

    if self.fill_var.get():
        draw.polygon(flat, fill=self._lighten(color), outline=color)
    else:

```

```

        draw.line(flat, fill=color, width=2)

self.photo = ImageTk.PhotoImage(self.img)
self.canvas.create_image(0, 0, image=self.photo, anchor=tk.NW)

# Подпись
self.canvas.create_text(
    cx, 20,
    text=f"Снежинка Коха | итераций: {iterations} | размер:
{size}",
    font=("Arial", 11), fill="gray"
)

@staticmethod
def _lighten(color_name: str) -> str:
    """Возвращает светлый оттенок для заливки."""
    mapping = {
        "blue": "#cce0ff",
        "black": "#cccccc",
        "red": "#ffd0d0",
        "green": "#ccffcc",
        "purple": "#e8ccff",
    }
    return mapping.get(color_name, "#e0e0ff")

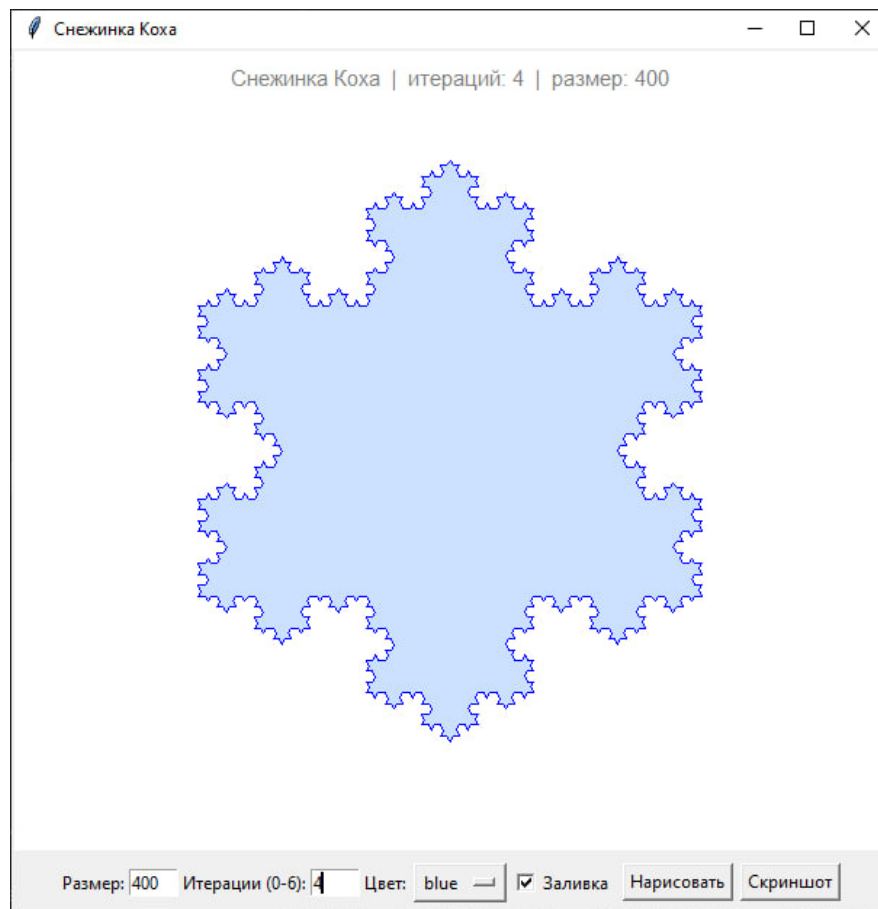
# ----- screenshot -----
--

def screenshot(self):
    self.canvas.update()
    x = self.canvas.winfo_rootx()
    y = self.canvas.winfo_rooty()
    x1 = x + self.canvas.winfo_width()
    y1 = y + self.canvas.winfo_height()
    filename = f"screenshot_koch_{int(time.time())}.png"
    ImageGrab.grab().crop((x, y, x1, y1)).save(filename)
    print(f"Скриншот сохранён: {filename}")

if __name__ == "__main__":
    main_root = tk.Tk()
    app = KochApp(main_root)
    main_root.mainloop()

```

Результат:



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.